INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	**INTRODUCCIÓN AL DESARROLLO WEB**				
TITULO DE LA PRÁCTICA:	*Fundamentos de JavaScript ECMAScript 6+, Manipulación del DOM y Manejo de Eventos*				
NÚMERO DE PRÁCTICA:	*04*	**AÑO LECTIVO:**	*2026*	**NRO. SEMESTRE:**	*2*
FECHA DE PRESENTACIÓN	*12/10/2026*	**HORA DE PRESENTACIÓN**	*19/00/00*		
INTEGRANTE (s) *Sarah Megan Ortiz Saavedra*				**NOTA (0-20)**	
DOCENTE(s): **MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.**					

RESULTADOS Y PRUEBAS

I. PROBLEMAS PROPUESTOS RESUELTOS:

Caso Práctico: Aplicación Interactiva de Gestión de Tareas Académicas

Desarrollar una aplicación interactiva "Gestor de Tareas Universitarias" en el navegador cliente sin refresco de página:

1. Gestión del Estado y Estructuras de Datos:

- *Mantener un arreglo de objetos JavaScript global para las tareas: [{ id, titulo, curso, fechaEntrega, completada }].*
- *Utilizar métodos iterativos ES6+ (map, filter, reduce, find) para la manipulación del arreglo.*

2. Captura y Validación con JavaScript:

- *Interceptar el evento submit del formulario de alta mediante preventDefault().*
- *Validar mediante JS que ningún campo esté vacío y que la fecha de entrega sea posterior a la fecha actual. En caso de error, mostrar mensajes dinámicos en un div de alertas.*

3. Manipulación Dinámica del DOM:

- *Renderizar la lista de tareas dinámicamente construyendo elementos DOM o usando template literals seguros.*
- *Permitir alternar el estado de la tarea (completada/pendiente), modificando sus clases CSS dinámicamente (ej. tachado de texto).*
- *Permitir la eliminación individual de una tarea del DOM y del arreglo.*
- *Implementar un filtro visual (Todas / Pendientes / Completadas) manipulando el DOM.*

4. Persistencia de Datos:

- *Sincronizar cada adición, cambio de estado o eliminación con localStorage.*
- *Cargar y renderizar las tareas guardadas automáticamente al desencadenarse el evento DOMContentLoaded.*

Index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Gestor de Tareas Universitarias</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/css/bootstrap.min.css"
rel="stylesheet">
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="contenedor">
        <form id="form-tareas">
            <h2> Tareas Universitarias</h2>
            <label for="titulo">Título</label>
            <input type="text" id="titulo" class="form-control">
            <label for="curso">Curso</label>
            <input type="text" id="curso" class="form-control">
            <label for="fecha-entrega">Fecha de Entrega</label>
            <input type="date" id="fecha-entrega" class="form-control">
            <button type="submit" class="btn btn-primary">Enviar</button>
        </form>
        <div id="alertas"></div>
        <div class="botonFiltro">
            <button type="button" id="btn-todas" class="btn btn-primary">Todas</button>
            <button type="button" id="btn-pendientes" class="btn btn-primary">Pendientes</button>
            <button type="button" id="btn-completadas" class="btn btn-primary">Completadas</button>
        </div>

        <p id="contador"></p>
        <ul id="lista-tareas" class="list-group"></ul>
    </div>
    <script src="app.js"></script>
</body>
</html>
```

App.js

```
const form = document.querySelector("#form-tareas");
```

```
const inputTitulo = document.querySelector("#titulo");
const inputCurso = document.querySelector("#curso");
const inputFecha = document.querySelector("#fecha-entrega");
const divAlertas = document.querySelector("#alertas");

const contadorTareas = document.querySelector('#contador');
const listaTareas= document.querySelector("#lista-tareas");

const btnTodas = document.querySelector("#btn-todas");
const btnPendientes= document.querySelector("#btn-pendientes");
const btnCompletadas= document.querySelector("#btn-completadas");


let tareas = [];
try {
   tareas = JSON.parse(localStorage.getItem("tareas")) || [];
}
catch (error) {
    tareas = [];
    console.log("Datos guardados dañados, empezando desde cero");
}

let filtro = "Todas";
function renderizarTareas(){
    listaTareas.innerHTML = "";
    let tareasVisibles = tareas;
    if (filtro === "Pendientes"){
        tareasVisibles = tareas.filter((t) => !t.completada);
    }
    if (filtro === "Completadas"){
        tareasVisibles = tareas.filter((t)=> t.completada)
    }

    tareasVisibles.forEach((t) => {
        const li = document.createElement("li");
        li.className = "list-group-item d-flex justify-content-between align-items-center";
        li.dataset.id = t.id;

        const span = document.createElement("span");
```

```
        span.textContent= `${t.titulo}  -  ${t.curso} (Entrega: ${t.fechaEntrega})`;
        span.classList.toggle("text-decoration-line-through", t.completada);

        const btnEliminar = document.createElement("button");
        btnEliminar.type = "button";
        btnEliminar.className = "btn btn-danger btn-sm";
        btnEliminar.textContent = "Eliminar";

        const btnCompletar = document.createElement("button");
        btnCompletar.type = "button";
        btnCompletar.className = "btn btn-success btn-sm me-2";
        btnCompletar.textContent = "Completar";

        li.appendChild(span);
        li.appendChild(btnCompletar);
        li.appendChild(btnEliminar);
        listaTareas.appendChild(li);
    });

    const cantidadPendientes = tareas.reduce((i, t) => {
        if (!t.completada){
            return i+1;
        }
        return i;
    }, 0);
    contadorTareas.textContent = `Pendientes: ${cantidadPendientes} de ${tareas.length}`;

}

function eliminarTarea (id) {
    tareas=tareas.filter(function(t){
        if (t.id !== id){
            return true;
        }
        else {
            return false;
        }
    });
```

```
    guardarTareas();
    renderizarTareas();
}

function completarTarea (id) {
    tareas=tareas.map(function(t) {
        if (t.id === id){
            t.completada = !t.completada;
        }
        return t;
    });
    guardarTareas();
    renderizarTareas();
}

function guardarTareas() {
    localStorage.setItem("tareas", JSON.stringify(tareas));
}

listaTareas.addEventListener("click", (e) =>{
    const elemento= e.target;
    if (elemento.classList.contains("btn-danger")){
        const li= elemento.closest("li");
        const id = Number(li.dataset.id);
        eliminarTarea(id);
    }

    if (elemento.classList.contains("btn-success")){
        const li= elemento.closest("li");
        const id = Number(li.dataset.id);
        completarTarea(id);
    }
});

form.addEventListener("submit", (evento) => {
    evento.preventDefault();
    const titulo = inputTitulo.value.trim();
    const curso= inputCurso.value.trim();
```

```
    const fechaEntrega = inputFecha.value;
    if (titulo === "" || curso==="" || fechaEntrega ===""){
        divAlertas.innerHTML = '<div class="alert alert-danger">Todos los campos son
obligatorios</div>';
        return;
    }

    const hoy = new Date();
    hoy.setHours(0,0,0,0);
    const fechaElegida = new Date(fechaEntrega + "T00:00:00");
    if (fechaElegida <= hoy){
        divAlertas.innerHTML = '<div class="alert alert-danger">La fecha de entrega debe ser
posterior a hoy</div>';
        return;
    }

    const repetida =tareas.find((t) => t.titulo === titulo && t.curso === curso);
    if (repetida){
        divAlertas.innerHTML = '<div class="alert alert-danger">Esa tarea ya existe</div>';
        return;
    }

    divAlertas.innerHTML = "";

    const tarea = {
        id: Date.now(),
        titulo: titulo,
        curso: curso,
        fechaEntrega: fechaEntrega,
        completada: false
    };

    tareas.push(tarea);
    guardarTareas();
    renderizarTareas();
    form.reset();
});
```

```
btnTodas.addEventListener("click", () => {
    filtro= "Todas";
    renderizarTareas();
});
btnPendientes.addEventListener("click", () => {
    filtro = "Pendientes";
    renderizarTareas();
});
btnCompletadas.addEventListener("click", () => {
    filtro = "Completadas";
    renderizarTareas();
});

document.addEventListener("DOMContentLoaded", renderizarTareas);
```

style.css

```
body {
    background-color: rgb(224, 236, 255);
}

.contenedor {
    font-family:'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    max-width: 700px;
    margin: 30px auto;
    padding: 20px;
    background-color: rgb(247, 250, 255);
    border-radius: 15px;
    box-shadow: 7px 12px 10px rgba(79, 80, 125, 0.532);
}

label {
    margin-top: 13px;
}

form button {
    margin-top: 15px;
}

.botonFiltro {
   display: flex;
```

```
    margin-top: 20px;
    gap: 13px;
}

#contador {
    margin: 20px 0;
    font-weight: bold;
}

#alertas {
    margin: 20px;
}
```



REPOSITORIO GITHUB: *https://github.com/mychemicalheart1712/IDWeb*
PROYECTO GITHUB: *https://github.com/mychemicalheart1712/IDWeb/tree/main/Lab04*

II. CUESTIONARIO:

1. ***¿Cuál es la diferencia de comportamiento entre las palabras clave var, let y const respecto al alcance (scope) y al hoisting?***
   - ***Alcance (Scope):*** *var tiene alcance de función la cual es visible en toda la función donde se declara e ignora los bloques (como if o for). Por el contrario, let y const tienen alcance de bloque, quedando limitadas al interior de las llaves {} donde fueron creadas*
   - ***Hoisting****: Con var, la variable se eleva y se inicializa automáticamente como undefined, por lo que usarla antes de su declaración no rompe el código, solo devuelve ese valor. En cambio, let y const se elevan pero no se inicializan , permanecen en la "zona muerta temporal" (Temporal Dead Zone) desde el inicio del bloque hasta la línea de su declaración, y al intentar usarlas antes de declararlas lanza un error de tipo ReferenceError.*

   Actualmente se utiliza const por defecto y let cuando el valor debe cambiar, var se evita porque su alcance de función y su hoisting facilitan errores difíciles de detectar.

2. ***Explique la técnica de Delegación de Eventos (Event Delegation) en el DOM y por qué es más eficiente que asignar listeners individuales a múltiples elementos.***

   La Delegación de Eventos es una técnica que consiste en asignar un único event listener a un elemento padre en lugar de ponérselo a cada uno de los elementos hijos.
   Esta técnica se basa en el Burbujeo de eventos (Event Bubbling), es decir, cuando haces clic en un elemento hijo, el evento "viaja" hacia arriba por el árbol DOM hasta llegar al padre. Luego usando event.target se puede identificar que hijo activó el evento

   Es más eficiente porque:
   - ***Ahorra memoria:*** *En vez de crear varios listeners (uno por cada elemento), solo creas uno solo para el contenedor principal*
   - ***Soporta elementos dinámicos****: Si agregas nuevos elementos hijos al DOM, funcionarán automáticamente sin necesidad de programarles nuevos listeners*
   - ***Código limpio:*** *Centralizas toda la lógica en un solo lugar, facilitando el mantenimiento del sistema web*

3. ***¿Por qué es necesario utilizar JSON.stringify() al almacenar objetos complejos en localStorage y qué limitaciones tiene este almacenamiento en términos de seguridad?***
   localStorage solo puede almacenar cadenas de texto, así que si se guarda un objeto directamente se convierte en [object Object] y se pierden los datos. Por eso JSON.stringify() convierte (serializa) el objeto en una cadena JSON que conserva su estructura de clave-valor, y luego JSON.parse() la convierte de nuevo en un objeto utilizable
   Limitaciones de seguridad: el almacenamiento local no debe utilizarse para guardar información confidencial o sensible debido a:
   - ***Falta de cifrado:*** *Los datos quedan expuestos.*
   - ***Vulnerabilidad a ataques XSS:*** *Facilidad de robo mediante cualquier script que este en la página.*
   - ***Ausencia de control de acceso:*** *Lo convierte en un lugar de riesgo para datos confidenciales.*

CONCLUSIONES

A través de este laboratorio comprendí cómo JavaScript hace dinámica una página: con el DOM se puede crear, modificar y eliminar elementos sin recargar; el patrón de la aplicación: cambiar el arreglo, guardarlo y volver a dibujar la lista. También entendí que la delegación de eventos permite usar un solo listener para botones creados dinámicamente, y que localStorage con JSON.stringify() y JSON.parse() conserva los datos al recargar, aunque no es seguro para información sensible.

METODOLOGÍA DE TRABAJO

1. *Armé la estructura HTML: formulario, filtros, contador, lista y div de alertas, cada uno con su id*
2. *Seleccioné los elementos con querySelector() y declaré el arreglo global de tareas*
3. *Maneje el Submit con preventDefault() y validar campos vacíos, fecha posterior a hoy y tareas repetidas (con find)*
4. *Rendericé la lista con createElement() y appendChild(), y completé (con map) y eliminé (con filter) tareas mediante delegación de eventos*
5. *Agregué el filtro Todas / Pendientes / Completadas y un contador con reduce*
6. *Guardé los datos en localStorage, los cargué con DOMContentLoaded, apliqué estilos con CSS externo y Bootstrap, y guardé mi avance con commits en GitHub*

REFERENCIAS Y BIBLIOGRAFÍA

Atuonwu, S. C. (2022, 10 de febrero). var, let y const: ¿Cuál es la diferencia? (K. Guardo Herrera, Trad.). freeCodeCamp. https://www.freecodecamp.org/espanol/news/var-let-y-const-cual-es-la-diferencia/

Kuwar, R. (2025, 18 de agosto). Event delegation: What it is and why it's useful. Medium. https://medium.com/@rohitkuwar/event-delegation-what-it-is-and-why-its-useful-59e983f87f97

Matharu, M. (2024, 17 de septiembre). When not to use local storage: Risks, examples, and secure alternatives. Medium. https://meenumatharu.medium.com/when-not-to-use-local-storage-risks-examples-and-secure-alternatives-de541fed56d2